

Secure Coding: Injection, XSS, Path Traversal

Injection (SQL)

Unsafe:

```
query = "SELECT * FROM users WHERE name = '" + userInput + "'"
```

User input: `' OR '1'='1'` → returns everything or bypasses login.

Safe:

```
query = "SELECT * FROM users WHERE name = ?"  
db.execute(query, [userInput]) // parameterized
```

The driver treats input as data, never SQL.

XSS (output encoding)

User input echoed into HTML is dangerous unless encoded. Context matters: HTML, attribute, JS, CSS, URL all encode differently. Rule: **encode on output**, in the receiver's context.

、 、 、

```
<div>HELLO <script>... // bad: raw insertion &lt;img src=x  
onerror=alert(1)&gt; // encoded presentation \` \` \`
```

Path traversal

Never splice user input into filesystem paths:

```
../ / ..\ ..%2f -> escape the web root and read /etc/passwd
```

Use a whitelist, canonicalize the resolved path, and ensure it stays under the base directory (path.resolve + startsWith check).

Input validation principles

Whitelist > blacklist – decide what is allowed, reject the rest

Validate **syntax** (type, length, charset) then **semantics** (domain logic)

Validate on the server, never trust client

Canonicalize before validating (unicode normalization, decoding)

Command injection

Never build shell commands from input. If unavoidable: use parameterized exec calls (execFile/spawn with array args), never a string concatenated into sh.

The single most reusable rule: **data and code must be separated**.
Wherever user data meets a parser (SQL, shell, HTML, LDAP, XML), choose the safe-parameterized path.

[← OWASP Top 10: The Threat Menu](#)

[SAST, DAST & SCA: The AppSec T...](#)